Understanding C2: from generated audio to parameter reliability

> Printable edition. Use the Contents links or PDF bookmarks to navigate. Long code lines wrap for printing; copy executable commands from the Markdown guide. Source-file links refer to the local repository; keep the three PDFs together for related-guide links.

Each local file link is followed by its full path as plain text. A final `:number` identifies the source line; in VS Code, open the file with `Ctrl+P` and jump to that line with `Ctrl+G`.

Evidence snapshot: 15 October 2026. Repository: `D:/VSCode/GitHub/Synth-Inversion-RP`.

Read this for understanding. Use the demonstration walkthrough (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/DEMONSTRATION_WALKTHROUGH.md`) for what to open and say, and the metrics reference (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`) for formulas and complete measured tables. Code blocks below are excerpts from the actual source, with clickable file and line links. They are explanations, not a sequence of commands to execute. Historical experiment commands are in V4_README.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/V4_README.md`).

Read sections 1-10 for the data/model foundations, 11-16 for the reliability experiment, and 17-23 for a real example, evidence, questions and file vocabulary. You do not need to memorize the code. Practise explaining what enters each block, what comes out, why it is there and how you checked it.

Contents

Choose a stage below. In VS Code, the **Outline** panel shows the same stage groups; expand the one you need. In Markdown preview, click a link to jump there. Original section numbers are retained.

Stage	What is inside
Part 1. Research overview	The research question and current completion status; sections 1-2
Part 2. Dataset generation	Synth setup, 13 parameters, Sobol sampling, rendering and data QA; sections 3-4
Part 3. Preprocessing and data splits	DC filtering, n/z coordinates, protected splits and log-mel features; sections 5-8
Part 4. Model training	Single CNN training, Gaussian ensemble training and uncertainty U; sections 9-10
Part 5. Inference and forward diagnostics	Frozen predictions, reconstruction R, perturbations S and step-size checks; sections 11-14
Part 6. Reliability calibration	Correctness labels, feature transforms, logistic fitting and q; section 15
Part 7. Evaluation and validation	Ablations, measured comparisons and paired bootstrap; section 16
Part 8. Real example and saved evidence	A complete clip walkthrough, EDA figures and scientific checks; sections 17-18
Part 9. Technology choices and justifications	Why these methods and what their limitations are; section 19
Part 10. Revision and code reference	Questions, one-minute explanation, file map and glossary; sections 20-23

For the practical demonstration, open the VS Code/browser walkthrough (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/DEMONSTRATION_WALKTHROUGH.md`). For metric formulas and all measured tables, open the metrics reference (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`).

Part 1. Research overview

1. What is my component trying to answer?

Imagine hearing a sound made by a machine with 13 knobs. You want to copy the sound, but you cannot see the original knob positions. A neural network guesses them. Some guesses will be good; others may be difficult to judge because that knob barely affects the sound in the current patch. C2 investigates how to tell the user which guesses deserve more trust.

You can explain it this way:

> My component estimates synthesizer settings from an isolated audio clip, then estimates the reliability of each predicted setting. I test whether rendering the predicted patch and making small controlled changes gives useful reliability information beyond the neural network's uncertainty alone.

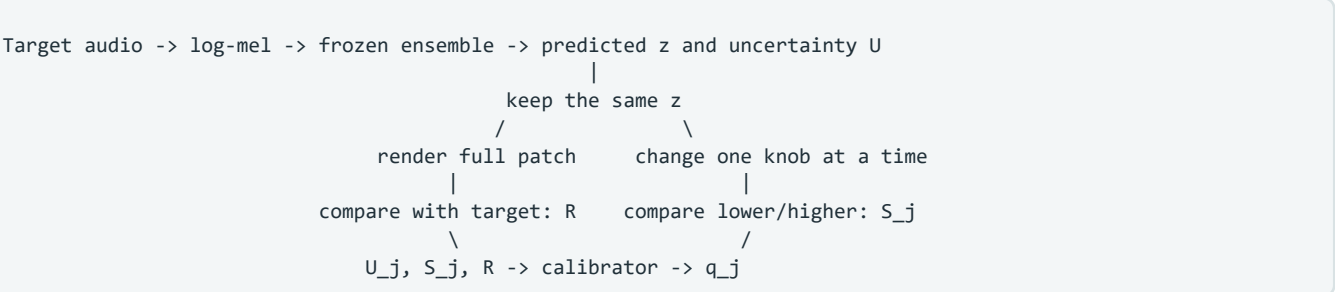
The research question is about **reliability of a fixed prediction**. A strong inverse model is needed, but inventing the best possible audio-to-synth network is not the main contribution. Neither a CNN, an ensemble nor a finite difference is new by itself. The proposed contribution is their controlled use in a synth-specific per-parameter correctness experiment, tested through ablations.

Three different questions must stay separate:

Question	Measurement
Were the original settings recovered?	Parameter MAE/RMSE/R2 and the within-tolerance label y
Does the reconstructed sound match under the chosen audio metric?	Reconstruction residual R
How likely is a particular setting to be within tolerance?	Calibrated correctness estimate q

Different settings can sometimes make the same or similar audio. For example, a filter envelope with zero depth cannot reveal its attack time through sound. Therefore a good-sounding reconstruction does not establish recovery of every original setting. Synthesizer non-uniqueness is also discussed in Hayes, Saitis and Fazekas; their multi-oscillator symmetry experiments do not prove the same failure mechanism for every control here.

After training, the information flows like this:



The original generating z teaches/evaluates this process on research data. It is not an input needed for the eventual reliability calculation on an unknown clip.

2. What has actually been completed?

Stage	Current evidence
Shared dataset	Completed 131,072 audio/parameter pairs, 13 controls, version v4
C2 partition and features	Frozen assignments; train-only feature statistics; full inverse-model feature cache
Single CNN	Full training complete; best epoch 53
Uncertainty models	Three Gaussian CNNs complete; best epochs 57, 59 and 58
Forward diagnostics	128 smoke, 2,048 calibration and 512 reliability-validation cases complete
Reliability fitting	All ablations complete at tolerances .025, .05 and .10
Follow-up checks	Per-control audit and fixed 256-clip step-size study complete
Browser demonstration	Six saved examples with real audio and measured scores
Live application	Upload, new-clip inference and patch-download interface pending
Final research evaluation	Method locking, untouched test and small human study pending

The intended application is: **isolated clip -> predicted settings and Vital patch -> reconstructed sound -> reliability information**. The saved viewer demonstrates outputs already produced by that research pipeline. It does not run a new prediction. The models currently cover the restricted preset and fixed recording conditions, not arbitrary notes, effects, synths or full music recordings.

Your proposal described a planned larger dataset and a roughly 10-16-control scope; its methodology also contains an older 65,536 reference. The implemented source of truth is now **131,072 and exactly 13**. Its main frozen-prediction, U/S/R and ablation ideas are implemented. Threshold display states and final test claims remain pending.

Do not confuse the datasets:

Name/location	Meaning
pilot_v1.csv (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/manifests/pilot_v1.csv`)	Historical 1,024-sound, eight-control engineering baseline
restricted_bend_sustain_v4.csv (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/manifests/restricted_bend_sustain_v4.csv`)	Earlier 1,024-sound prefix of the new 13-control family
corpus_v4/manifest.parquet (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/raw/corpus_v4/manifest.parquet`)	Canonical complete 131,072-sound dataset used by current C2
protocol_v1/manifest.parquet (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/protocol_v1/manifest.parquet`)	C2's derived manifest: canonical labels, z coordinates and assignments

The old pilot's 80/10/10 split and eight-control test scores are historical. They are not the partition or test results of today's C2 experiment.

Back to contents

Part 2. Dataset generation

3. What exactly makes one sound?

A synthesizer is the forward machine: settings go in, audio comes out. An inverse model learns the harder direction: audio goes in, predicted settings come out. **Vital** is the synthesizer; **Vita** is the Python wrapper controlling its engine. The CNN itself does not synthesize waveform samples.

The fixed recipe uses one oscillator with an embedded Basic Shapes wavetable, Bend mode, an Analog 12 dB filter, amplitude ADSR and ENV2 routed to cutoff. Other oscillators, noise and effects stay off. Note, velocity, phase, routing and envelope curve settings remain fixed so changing a sample means changing the declared controls.

Recording setting	Value and reason
MIDI note	60, middle C; pitch is held constant to study the selected timbral controls
Velocity	.8; avoids adding another changing cause of audio differences
Note held	1.5 seconds; observes attack, decay and held behaviour
Full render	3 seconds; includes note-off and release
Sample rate	44,100 Hz; one fixed rate throughout the corpus and C2
Channel count	Canonical mono; engine stereo equality is checked before taking one channel
Samples per clip	44,100 x 3 = 132,300
Stored format	Raw FLOAT WAV masters; conditioning is derived when read

An oscillator supplies the repeating waveform. A filter changes its spectrum. Attack describes the rise, decay the move toward the held level, sustain the held level, and release the fall after note-off. ENV2 changes the filter through time; the amplitude envelope changes the volume through time.

The 13 parameters and their actual controls

The bounds below are **Vital normalized values n**, not seconds or Hz. Model z values rescale each restricted interval to 0-1.

Parameter	Engine control	Allowed n	Plain meaning
wave_frame	osc_1_wave_frame	0-1	Position within the fixed wavetable
bend_amount	osc_1_distortion_amount	.25-.75	Bend waveform shaping; .5 is neutral
filter_cutoff	filter_1_cutoff	.30-.85	Main filter frequency control
filter_resonance	filter_1_resonance	0-.80	Emphasis around the cutoff region
filter_drive	filter_1_drive	0-.75	Nonlinear filter drive
amp_attack	env_1_attack	.05-.35	How quickly volume rises
amp_decay	env_1_decay	.15-.40	How quickly volume moves to sustain
amp_sustain	env_1_sustain	.20-1	Held amplitude-envelope level
amp_release	env_1_release	.10-.45	Volume fall after note-off

Parameter	Engine control	Allowed n	Plain meaning
filter_env_amount	modulation_1_amount	.50-.5625	Depth of ENV2's cutoff movement; n=.5 means zero depth
filter_env_attack	env_2_attack	.05-.35	Rise of the filter envelope
filter_env_decay	env_2_decay	.15-.40	Move toward its held level
filter_env_sustain	env_2_sustain	0-1	Held filter-envelope fraction

The inheritance chain is params_config.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/params_config.py`) -> restricted_config.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/restricted_config.py`) -> bend_config.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/bend_config.py`) -> bend_sustain_config.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/bend_sustain_config.py:28`). The v4 file adds variable filter-envelope sustain back to the Bend architecture. The frozen preset is base_restricted_bend_sustain_v4.vital (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/presets/base_restricted_bend_sustain_v4.vital`).

Why 13? The selected set keeps both oscillator waveform shaping and the held filter trajectory while keeping the experiment manageable. Thirteen is not intrinsically optimal. Ranges were chosen through engineering characterization and researcher review, not a completed human validation of independent perceptual effects. The dated decision is 2026-10-07_bend_and_sustain.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/decisions/2026-10-07_bend_and_sustain.md`). That record preserves what was pending then; later integration is evidenced by the current policy, recipe and completed corpus, not by rewriting the old record.

Unit details are in PARAMETER_UNITS.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/PARAMETER_UNITS.md`). Cutoff's native value is a pitch coordinate, envelope-time native values use a quartic mapping, and drive has a dB mapping. Do not call every normalized number Hz, seconds or a percentage of perceived sound. The saved units audit checked 221 control points and 17 preset serialization roundtrips; it did not measure acoustic response at those points.

4. How was the dataset generated and selected?

4.1 Choose parameter combinations with Sobol

Imagine placing dots across a box rather than letting many dots accidentally bunch in one corner. Sobol supplies a reproducible, low-discrepancy design in the 13 parameter coordinates. The full design has 131,072 = 2^17 points. Generation uses seed **20260803**, distinct from C2's later partition/training seed.

generation/scripts/05_generate_pilot_dataset.py:206 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:206`)

```
def create_sobol_samples(
    sample_count: int,
    dimensions: int,
    seed: int,
) -> np.ndarray:
    """
    Generate a balanced Sobol sequence.

    random_base2 requires a power-of-two sample count, so the script
    creates the next power of two and then keeps only the requested rows.
    """
    if sample_count <= 0:
        raise ValueError("Sample count must be greater than zero.")

    if dimensions <= 0:
        raise ValueError("At least one parameter is required.")
```

```
    power = math.ceil(math.log2(sample_count))

    sampler = qmc.Sobol(
    d=dimensions,
    scramble=True,
    rng=np.random.default_rng(seed),
    )

    samples = sampler.random_base2(m=power)

    return samples[:sample_count]
```

`d` is the number of knobs; scrambling uses a fixed random generator; `random_base2` draws a power-of-two design. SciPy documents the balance property for that full design and warns about dropping/thinning points. A random train subset should not be described as preserving every balance guarantee of the full Sobol net. Official Sobol documentation.

Sobol improves coverage in the chosen coordinates. It does not establish uniform perceived timbre coverage, independent audible effects, or coverage of all Vital sounds. Uniform normalized time controls also do not mean uniform durations in seconds.

4.2 Set controls, then record what the engine actually accepted

There are four different quantities:

Quantity	Meaning
`*_sample_01`	Proposed Sobol location inside the restricted interval
`*_normalized`	Actual normalized value read back from the engine; canonical label
`*_raw`	Actual native control value
`*_display`	Human-readable rounded text; useful for inspection, not training truth

The sampler requests `n = lower + sample_01 * (upper-lower)`. The code sets that value and reads back the control. Tiny floating-point differences are retained.

generation/scripts/05_generate_pilot_dataset.py:354 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:354`)

```
def apply_sampled_parameters(
    synth: vita.Synth,
    unit_sample: np.ndarray,
    parameter_names: list[str],
) -> dict[str, Any]:
    controls = synth.get_controls()
    recorded_values: dict[str, Any] = {}

    for dimension, parameter_name in enumerate(parameter_names):
        config = PARAMS[parameter_name]

        minimum = float(config["min"])
        maximum = float(config["max"])

        unit_value = float(unit_sample[dimension])

        normalized_value = (
            minimum
            + unit_value * (maximum - minimum)
        )

        control_name = str(config["control"])
        mode = str(config["mode"])
```

```
        set_control(
            controls=controls,
            name=control_name,
            mode=mode,
            value=normalized_value,
        )

        control = controls[control_name]

        recorded_values[f"{parameter_name}_sample_01"] = (
            unit_value
        )

        recorded_values[f"{parameter_name}_normalized"] = (
            float(control.get_normalized())
        )

        recorded_values[f"{parameter_name}_raw"] = (
            float(control.value())
        )

        recorded_values[f"{parameter_name}_display"] = (
            str(synth.get_control_text(control_name))
        )

    return recorded_values
```

The `set_control` call applies the coordinate with the declared setter. The four recorded fields preserve design, engine state and display separately. Ground truth comes from **the synthesizer that generated the clip**, not a human annotation or another model. Labels alone are not waveforms; join audio and labels by `sample_id`.

4.3 Render each complete patch

render_one (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py:225`) creates a fresh synth, loads the preset, applies fixed settings, applies one sampled vector, renders it and records identifiers, controls, audio QA and sample hashes. render_sample (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:406`) calls `synth.render(MIDI_NOTE, VELOCITY, NOTE_DURATION, RENDER_DURATION)` and checks shape and finite values. A fresh instance and fixed phase reduce state/randomness confounds between clips.

ordered_results (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py:250`) runs workers concurrently but commits in sample-index order. save_master (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py:278`) saves raw FLOAT WAVs atomically; commit (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py:293`) checkpoints 128 new rows. The runner verifies an imported prefix and checks existing files on resume rather than overwriting conflicting audio.

The full manifest is manifest.parquet (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/raw/corpus_v4/manifest.parquet`) or its CSV copy (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/raw/corpus_v4/manifest.csv`). Its `raw_audio_path` is authoritative: imported prefix audio can remain in its original folder, so not all 131,072 files necessarily live in one directory. recipe.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/raw/corpus_v4/recipe.json`) records preset/source/runtime identities and conditions.

4.4 Check audio quality and integrity

The completed full-corpus verification (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/team_handoff_v4_131072/verification.json`) records 131,072 verified rows and unique raw paths, zero uncheckpointed WAVs, verified raw/conditioned hashes, matching CSV/Parquet/checkpoints and zero conditioned silence/clipping/DC warnings. Generation **flags** these conditions; this is not a claim that bad Sobol rows were silently removed or resampled.

The generation QA in calculate_audio_metrics (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:448`) uses silence below -60 dBFS RMS, clipping at peak >=1, and a DC warning at absolute mean >=.005 or absolute mean/RMS >=.20. A zero warning count means passing those declared rules, not all possible perceptual quality tests.

audit_full_corpus.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audit_full_corpus.py:69`) rechecks files, waveforms, labels, display strings and QA against checkpoints. Hash identity and re-render equality are different: a hash checks saved bytes; an independent known-parameter roundtrip checks whether the synthesis path can reproduce them. Source/data hashes provide traceability, not scientific validity alone.

The full corpus was generated earlier. This guide does not regenerate it. The latest generation summary's elapsed time describes its last resumed invocation, not necessarily total time spent across every generation session.

Back to contents

Part 3. Preprocessing and data splits

5. Why condition audio before extracting features?

DC offset means the waveform is displaced from its zero centre. The selected policy applies the same causal second-order Butterworth 10 Hz high-pass to every clip. It is an engineering choice supported by the saved numerical checks and informal researcher listening, not a formal perceptual equivalence study.

generation/audio_processing.py:30 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audio_processing.py:30`)

```
def dc_filter(audio, sample_rate: int, *, input_policy_id: str = "raw") -> np.ndarray:
    if input_policy_id != "raw":
        raise ValueError("DC filtering requires raw input; refusing repeat processing")
    definition = policy_definition()["policy"]
    if sample_rate != definition["sample_rate_hz"]:
        raise ValueError("DC policy requires canonical 44100 Hz audio")
    return sosfilt(np.asarray(definition["filter"]["sos"]), mono_float(audio))
```

The policy is dc_highpass_10hz_v1.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audio_policies/dc_highpass_10hz_v1.json`). There is no per-clip peak/loudness normalization in C2. This matters because amplitude, drive and envelope changes are part of the signal we want to inspect. The repository also contains level-matching helpers for other review workflows; their presence does not mean the C2 loader applies them.

generation/corpus_audio.py:48 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/corpus_audio.py:48`)

```
def load_audio(row, root=ROOT, *, verify=False):
    """Always apply DC once to RAW; cast as the historical processed FLOAT WAV did.

    Use this adapter before feature extraction/training instead of treating
    raw_audio_path as processed audio. No persistent second WAV is needed.
    """
    if row["audio_policy_id"] != DC_POLICY_ID:
        raise ValueError("Unexpected audio policy")
    raw = read_raw(row, root)
    processed = canonical_float32(dc_filter(raw, 44100))
    if verify:
        if sample_digest(raw) != row["raw_samples_sha256"]:
            raise ValueError(f"Raw waveform mismatch: {row['sample_id']}")
        if sample_digest(processed) != row["processed_samples_sha256"]:
            raise ValueError(f"Processed waveform mismatch: {row['sample_id']}")
    return processed, 44100
```

The loader always begins with a raw master and applies conditioning once. During feature extraction `verify=True` checks both raw and conditioned sample hashes. The processed waveform is derived in memory, avoiding a second full stored corpus.

canonical_float32 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/corpus_audio.py:12`) sets only float32 subnormal values to signed zero so CPU flush-to-zero state does not change hashes of inaudibly tiny tails. Raw masters remain unchanged. This is a reproducibility detail, not denoising or loudness matching.

6. What are n and z, and why rescale targets?

Different knobs have different allowed interval widths. Training directly on them would give a wide-range knob more influence simply because its numbers move farther. Each actual normalized readback n is mapped to `z=(n-lower)/(upper-lower)`.

c2_estimation/v4/data.py:56 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:56`)

```
def to_z(values: np.ndarray, lower: np.ndarray, upper: np.ndarray) -> np.ndarray:
    values = np.asarray(values, dtype=np.float64)
    z = (values - lower) / (upper - lower)
    # Preserve readbacks, including tiny floating-point excursions; never silently clip labels.
    if not np.isfinite(z).all() or np.any(z < -2e-6) or np.any(z > 1 + 2e-6):
        raise ValueError("Readback is outside the restricted research interval")
    return z
```

For cutoff, lower=.30 and upper=.85. If z=.40, the normalized Vital value is `.30 + .40*.55 = .52`. A z error of .05 corresponds to .0275 in that normalized control, but not a constant Hz error. This is a coordinate example, not a measured model prediction. Reconstruct using from_z (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:65`).

z uses the **actual readback**, not the requested Sobol proposal. The generator's sampling coordinate is sometimes called q in older unit notes; it is unrelated to C2's calibrated correctness probability q.

7. How are clips separated without leakage?

Think of separate exercise books: learn the settings from one, learn the reliability rule from another, choose settings using development books, and keep the final exam sealed. Saving assignments makes the same examples keep the same role.

Partition	Pool size	Actual use so far
Train	98,304 (75%)	Fit CNN weights; train-only normalization; 128 smoke diagnostics
Calibration	16,384 (12.5%)	Predetermined 2,048 clips fit the reliability models
Validation / inverse	4,096 (3.125%)	Choose CNN checkpoints and inspect inverse errors
Validation / reliability	4,096 (3.125%)	Predetermined 512 choose calibrator regularization and provide development comparisons
Test	8,192 (6.25%)	Reserved; 1,024 diagnostic IDs assigned but not evaluated

The two validation roles total 8,192. These are four main partitions, with validation split into two disjoint roles. The first 1,024 v4 clips had already been inspected, so they are forced into training. A seeded permutation assigns the rest. Seed **20261013** controls this protocol; a separate seed+1 ordering selects diagnostic subsets before errors, uncertainty or audio quality of predictions are observed.

c2_estimation/v4/data.py:72 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:72`)

```
def assignments(count: int, *, seed: int = SEED, counts: dict | None = None,
                prefix_count: int = 1024, diagnostic_counts: dict | None = None) -> pd.DataFrame:
```

```
    counts = COUNTS if counts is None else counts
    diagnostic_counts = DIAGNOSTIC_COUNTS if diagnostic_counts is None else diagnostic_counts
    if set(counts) != set(COUNTS) or sum(counts.values()) != count:
        raise ValueError("Four disjoint split sizes must sum to the sample count")
    if not 0 <= prefix_count <= counts["train"] or counts["validation"] % 2:
        raise ValueError("Invalid development prefix or odd validation count")
    # The already studied v4 pilot is training-only. All remaining assignments are random.
    rng = np.random.default_rng(seed)
    remaining = rng.permutation(np.arange(prefix_count, count))
    splits = np.full(count, "train", dtype="<U11")
    offset = counts["train"] - prefix_count
    for name in ("calibration", "validation", "test"):
        splits[remaining[offset:offset + counts[name]]] = name
        offset += counts[name]
    roles = np.full(count, "", dtype="<U11")
    val = rng.permutation(np.flatnonzero(splits == "validation"))
    roles[val[:len(val) // 2]] = "inverse"
    roles[val[len(val) // 2:]] = "reliability"
    rank = np.full(count, -1, dtype=np.int64)
    diagnostic_rank = np.full(count, -1, dtype=np.int64)
    subset_rng = np.random.default_rng(seed + 1)
    for name in counts:
        eligible = np.flatnonzero(splits == name)
        ordered = subset_rng.permutation(eligible)
        rank[ordered] = np.arange(len(ordered))
        diagnostic_pool = ordered[roles[ordered] == "reliability"] if name == "validation" else ordered
        wanted = diagnostic_counts[name]
        if not 0 <= wanted <= len(diagnostic_pool):
            raise ValueError("Diagnostic subset is larger than its eligible partition")
        diagnostic_rank[diagnostic_pool[:wanted]] = np.arange(wanted)
    return pd.DataFrame({"split": splits, "validation_role": roles,
                         "selection_rank": rank, "diagnostic_rank": diagnostic_rank})
```

prepare (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:132`) verifies agreement with the shared parameter reference and saves protocol.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/protocol_v1/protocol.json`), split_manifest.csv (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/protocol_v1/split_manifest.csv`) and the derived manifest. Hashes protect later reuse.

Important limits: disjoint IDs and no duplicate complete parameter vectors do not guarantee every pair is acoustically distinct or all neighbouring patches are far apart. The intended evaluation is within this sampling domain, not a held-out synthesizer-family or instrument-generalization test. Derived perturbations stay with their source partition and never become inverse-training examples.

Full-corpus integrity/format checking includes files later assigned to test. That is different from extracting test features, measuring a model's test predictions or using test outcomes to choose a method. The former checks the dataset exists correctly; the latter remain reserved for the locked final evaluation.

8. How does audio become an input image?

A waveform lists pressure-like sample values over time. A spectrogram shows which frequencies are present at each time. The mel filterbank groups frequencies into 64 broad bands; the logarithm compresses the scale so quiet and loud structure can both be represented. The CNN receives numbers, not a rendered PNG image.

The exact sequence is: canonical waveform -> reflect padding -> overlapping Hann windows -> real FFT -> magnitude scaling -> power -> triangular mel bands -> dB -> fixed -80 to 0 dB clipping -> float32 array.

c2_estimation/pipeline.py:323 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:323`)

```
def extract_log_mel(
    audio: np.ndarray,
    window: np.ndarray,
    mel_filterbank: np.ndarray,
```

```
    n_fft: int,
    hop_length: int,
    top_db: float,
) -> np.ndarray:
    padded = np.pad(audio.astype(np.float32, copy=False), n_fft // 2, mode="reflect")
    frames = np.lib.stride_tricks.sliding_window_view(padded, n_fft)[::hop_length]
    spectrum = np.fft.rfft(frames * window, n=n_fft, axis=1)
    amplitude = np.abs(spectrum) / max(float(window.sum() / 2.0), np.finfo(float).eps)
    power = np.square(amplitude)
    mel_power = power @ mel_filterbank.T
    log_mel_db = 10.0 * np.log10(np.maximum(mel_power, 1e-10))
    log_mel_db = np.clip(log_mel_db, -top_db, 0.0)
    return np.ascontiguousarray(log_mel_db.T, dtype=np.float32)
```

Setting	Value / reason
FFT window	2,048 samples, about 46.44 ms
Hop	512 samples, about 11.61 ms
Frequency limits	20-20,000 Hz, below the 22,050 Hz Nyquist frequency
Mel bands	64, a compact time/frequency representation
Clip length	3 seconds -> 259 frames with this padding/hop
Per-clip array	64 x 259; model adds a single input channel
dB reference	Fixed 0 dBFS; not each clip's own maximum

create_mel_filterbank (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:300`) constructs the triangular filters. features.build_cache (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/features.py:28`) checks allowed IDs and hashes, extracts features and saves 512-clip float32 shards. The full cache contains 98,304 train + 4,096 inverse-validation clips, about 6.3 GiB. It is memory-mapped during training rather than loaded as one enormous tensor.

We then standardize with one scalar mean and standard deviation computed over training spectrogram values only:

c2_estimation/v4/features.py:117 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/features.py:117`)

```
def training_statistics(arrays: list[np.ndarray], train_indexes: np.ndarray) -> tuple[float, float]:
    total, square, count = 0., 0., 0
    for number, array in enumerate(arrays):
        local = train_indexes[(train_indexes >= number * SHARD_SIZE) &
                              (train_indexes < number * SHARD_SIZE + len(array))] - number * SHARD_SIZE
        for start in range(0, len(local), 64):
            block = np.asarray(array[local[start:start+64]], dtype=np.float64)
            total += float(block.sum())
            square += float(np.square(block).sum())
            count += block.size
    if count == 0:
        raise ValueError("Cannot fit normalization without training samples")
    mean = total / count
    return mean, max(float(np.sqrt(max(square / count - mean * mean, 0.))), 1e-6)
```

The saved full-training scalar mean is -73.5395297540483 and standard deviation 12.9035521819895, before standardizing log-mel values. The model input is `(log_mel - training_mean)/training_std`. This is feature standardization, not per-audio loudness normalization. Calibration/validation/test do not supply new normalization statistics.

Feature settings and identity are in features_full_v1/cache.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/features_full_v1/cache.json`). PNG spectrograms are explanatory visualizations of arrays, saved by save_examples (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/baseline.py:84`).

Back to contents

Part 4. Model training

9. How does the single CNN learn the settings?

A convolution slides small filters over the spectrogram and learns useful local patterns. ReLU introduces nonlinearity. Pooling compresses position detail. The final dense layers combine those patterns into 13 continuous outputs. This is regression, not classification into 13 sound categories.

c2_estimation/pipeline.py:82 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:82`)

```
class ParameterCNN(nn.Module):
    """Compact CNN whose outputs are normalized synthesizer parameters."""

    def __init__(self, target_minimums: np.ndarray, target_maximums: np.ndarray) -> None:
        super().__init__()
        output_count = len(target_minimums)
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((2, 4)),
        )
        self.regressor = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 2 * 4, 64),
            nn.ReLU(),
            nn.Linear(64, output_count),
        )
        minimums = torch.as_tensor(target_minimums, dtype=torch.float32)
        ranges = torch.as_tensor(target_maximums - target_minimums, dtype=torch.float32)
        self.register_buffer("target_minimums", minimums)
        self.register_buffer("target_ranges", ranges)

    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        logits = self.regressor(self.features(inputs))
        return self.target_minimums + torch.sigmoid(logits) * self.target_ranges
```

The v4 builder supplies 13 lower bounds of zero and upper bounds of one, so the sigmoid outputs are research z values. Merely importing this class from the pilot module does not make current training an eight-control pilot run.

```
1 x 64 x 259
  -> convolution 8 channels + max pool: 8 x 32 x 129
  -> convolution 16 channels + max pool: 16 x 16 x 64
  -> convolution 32 channels + max pool: 32 x 8 x 32
  -> average pool: 32 x 2 x 4
  -> flatten 256 -> dense 64 -> dense 13 -> sigmoid
```

The single model has **23,181 trainable parameters**. The CUDA-specific build_model (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/baseline.py:27`) uses a fixed 4x8 average pool instead of adaptive 2x4 pooling for this exact input shape. Values/gradients were checked to agree; the change permits deterministic CUDA backward execution.

The training loop is in baseline.train (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/baseline.py:99`). Its central operations are:

```
optimizer.zero_grad(set_to_none=True)
prediction = model(x.to(device))
loss = torch.mean(torch.square(prediction - y.to(device)))
loss.backward()
optimizer.step()
```

The model predicts, MSE measures disagreement with true z, backpropagation computes weight gradients, and Adam updates weights. The forward synthesizer is not in this gradient path. Training does not optimize R or a perceptual audio loss.

Full training used 98,304 clips, Adam learning rate .001, weight decay .0001, batch 128, 60 maximum epochs and patience 12. An epoch means one pass through the training set. Early stopping counts consecutive epochs without inverse-validation improvement; all 60 ran in this experiment. Epoch 53 had lowest inverse-validation MSE and supplies the best checkpoint. The last checkpoint also stores optimizer/history for resume.

Reproducibility uses fixed seeds, deterministic algorithms, deterministic epoch shuffle, disabled TF32, hashes and exact checkpoint-reload checks. It does not promise bit-identical training on every CPU, GPU, OS or dependency version.

Measured macro z MAE is **.126112**, and z MSE is **.031867**, versus **.083213** for always predicting training-set means. The matched original 512-clip comparison reduced MAE from .184187 to .126339; this changes multiple training conditions, so it is not a causal experiment on dataset size alone.

Open baseline_full_v1/summary.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/summary.json`), validation_metrics.csv (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/validation_metrics.csv`) and learning_curves.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/learning_curves.png`).

10. Where does uncertainty U come from?

Think of asking three independently trained estimators. Each gives a setting and a spread. We retain both their own predicted spread and disagreement between their means. Agreement alone is not proof of correctness: all three can make the same error.

Each Gaussian CNN keeps the convolutional/dense feature extractor, then uses a sigmoid mean head and a softplus variance head plus 1e-4. It has **24,026 trainable parameters**. The variance is positive and input-dependent; it is not yet a calibrated correctness probability.

c2_estimation/v4/ensemble.py:29 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:29`)

```
class GaussianCNN(nn.Module):
    def __init__(self, device, variance_floor=1e-4):
        super().__init__()
        if not 0 < variance_floor < .05:
            raise ValueError("Variance floor must be in (0, .05)")
        base = build_model(device)
        self.features = base.features
        self.shared = nn.Sequential(*list(base.regressor.children())[:-1])
        self.mean_head = nn.Linear(64, 13)
        self.variance_head = nn.Linear(64, 13)
        # Start near the variance of a moderately informative bounded regressor.
        nn.init.zeros_(self.variance_head.weight)
        nn.init.constant_(self.variance_head.bias, float(np.log(np.expm1(.05-variance_floor))))
        self.variance_floor = variance_floor

    def forward(self, x):
        hidden = self.shared(self.features(x))
        mean = torch.sigmoid(self.mean_head(hidden))
        variance = torch.nn.functional.softplus(self.variance_head(hidden)) + self.variance_floor
        return mean, variance
```

c2_estimation/v4/ensemble.py:51 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:51`)

```
def gaussian_nll(mean, variance, target):
    return .5 * (torch.log(variance) + (mean-target).square()/variance).mean()
```

The Gaussian negative-log-likelihood balances squared error divided by predicted variance against log variance. Predicting enormous variance has a cost; predicting tiny variance for large errors also has a cost. The constant .5*ln(2*pi) is omitted because it does not change the training gradients. NLL may be negative for small continuous variances; that alone does not indicate a bug.

This is a working regression objective. Although means are bounded, an ordinary Gaussian has unbounded support; this is not a truncated posterior or proof of correct uncertainty. The three seeds are 20261013, 20262022 and 20263031. Each member is selected by its own lowest inverse-validation **NLL**, not baseline MSE.

c2_estimation/v4/ensemble.py:55 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:55`)

```
def combine(means, variances):
    """Equal-weight mixture moments; population (ddof=0) between-member variance."""
    means, variances = np.asarray(means, dtype=np.float64), np.asarray(variances, dtype=np.float64)
    if means.ndim != 3 or means.shape != variances.shape or means.shape[-1] != 13:
        raise ValueError("Expected member x sample x 13 matching moments")
    if not np.isfinite(means).all() or not np.isfinite(variances).all() or (variances <= 0).any():
        raise ValueError("Invalid probabilistic predictions")
    mean = means.mean(axis=0)
    within = variances.mean(axis=0)
    between = np.square(means-mean).mean(axis=0)
    return mean, within, between, within+between
```

In notation, `mean=average(mu_m)`, `within=average(v_m)`, `between=average((mu_m-mean)^2)` and **U=within+between** in z squared. The between term uses population averaging, not an n-1 sample-variance denominator. Terms are operational summaries; do not present them as proven pure aleatoric/epistemic parts. The deep ensembles paper motivates this practical family of uncertainty methods; our own validation determines its performance here.

On the same 4,096 inverse-validation clips, ensemble MAE is **.127991**, slightly worse overall than the single CNN's .126112. Some controls improve, others worsen. Do not claim the ensemble improves everything or that three is the optimal member count. The point of retaining it is to test its uncertainty information separately.

Weights and provenance are in ensemble_full_v1/run_config.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/run_config.json`), summary.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/summary.json`) and `member_00`, `member_01`, `member_02` subfolders, each with `best_model.pt` and `last_model.pt`. The complete ensemble needs all three best checkpoints and the recorded preprocessing configuration.

Back to contents

Part 5. Inference and forward diagnostics

11. How do we freeze predictions for the reliability experiment?

predict_subsets.predict (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/predict_subsets.py:42`) loads the frozen ensemble, verifies model/source/protocol identities, uses the same audio and log-mel rules, and applies training normalization without refitting it. Each batch is predicted twice to check repeat inference. Means/variances and IDs are exported before any forward diagnostic is generated.

c2_estimation/v4/predict_subsets.py:27 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/predict_subsets.py:27`)

```
def select(frame, partition):
    eligible = frame[frame.diagnostic_rank >= 0]
    if partition == "smoke":
        selected = eligible[eligible.split == "train"]
    elif partition == "calibration":
        selected = eligible[eligible.split == "calibration"]
    elif partition == "reliability-validation":
        selected = eligible[(eligible.split == "validation") & (eligible.validation_role == "reliability")]
    else:
        raise ValueError("Only smoke, calibration and reliability-validation are permitted; test stays
            locked")
    if selected.empty or (selected.split == "test").any():
        raise ValueError("Invalid diagnostic selection")
    return selected.sort_values("diagnostic_rank").reset_index(drop=True)
```

The allowed partitions are smoke, calibration and reliability-validation. This command rejects test. The calibrator sees only U/S/R; true z exists in research records for labels/evaluation but never becomes a reliability input.

Frozen records are under predictions_calibration/predictions.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/predictions_calibration/predictions.json`) and predictions_reliability-validation/predictions.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/predictions_reliability-validation/predictions.json`). The corresponding Parquet files hold actual/predicted z and uncertainty components.

12. What is reconstruction residual R?

Take the predicted 13 settings, map them to normalized Vital controls and render a new clip using the same preset/note/phase conditions. Compare this clip with the target. That difference is R. R is a clip-level number shared by the 13 per-control calibrators; it does not identify which individual parameter caused a mismatch.

Renderer.render (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:90`) creates a fresh synth, loads the frozen preset, applies controls, checks readbacks, renders, verifies stereo agreement and applies canonical conditioning. representation (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:32`) uses linear-frequency STFT magnitudes at FFT sizes 512/1024/2048 with quarter-window hops. Short windows emphasize timing detail; longer windows resolve frequency more finely. All three contribute to one fixed diagnostic distance.

c2_estimation/v4/diagnostics.py:44 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:44`)

```
def feature_distance(a, b):
    if len(a) != 3 or len(b) != 3:
        raise ValueError("Expected three STFT resolutions")
    terms = []
    for first, second in zip(a, b, strict=True):
        if first.shape != second.shape or not np.isfinite(first).all() or not np.isfinite(second).all():
            raise ValueError("Incompatible or nonfinite STFT representations")
        sc = np.linalg.norm(first-second) / (.5*(np.linalg.norm(first)+np.linalg.norm(second)) + 1e-12)
        log_difference = np.abs(np.log(np.maximum(first, 1e-4)) - np.log(np.maximum(second, 1e-4))).mean()
        terms.append({"spectral_convergence": float(sc), "log_magnitude": float(log_difference)})
    return {"distance": float(np.mean([t["spectral_convergence"]+t["log_magnitude"] for t in terms])),
            "resolutions": terms}
```

At each resolution, spectral convergence measures magnitude-pattern error relative to average magnitude norm. Log-magnitude difference measures average absolute log disagreement, with a fixed amplitude floor of 1e-4. D averages SC+LM over resolutions. The denominator is symmetric in both sounds; the 1e-12 epsilon is distinct from the -80 dB magnitude floor. R is `D(target,predicted_render)`.

Parallel WaveGAN provides precedent for using multiple STFT resolutions in audio generation. It does not validate our exact symmetric distance as a perceptual similarity scale. R=.10 does not mean 90% similarity. Listening is useful, but a formal perceptual claim needs its own study.

13. How are perturbations made, and what is S?

For each predicted patch, move one knob a little lower and a little higher while all other settings stay fixed. Render both versions. If they differ strongly, that knob is locally acoustically active under this metric. Divide by the actual distance between knob positions so the score accounts for step size.

c2_estimation/v4/diagnostics.py:62 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:62`)

```
def perturbation_pair(z, parameter, delta=.02):
    z = np.asarray(z, dtype=np.float64)
    if z.shape != (13,) or not np.isfinite(z).all() or np.any(z < 0) or np.any(z > 1):
        raise ValueError("Expected 13 bounded research coordinates")
    if not 0 < delta <= .5 or not 0 <= parameter < 13:
        raise ValueError("Invalid step or parameter")
    minus, plus = z.copy(), z.copy()
    if z[parameter] < delta:
        plus[parameter] += delta
        mode = "forward"
    elif z[parameter] > 1-delta:
        minus[parameter] -= delta
        mode = "backward"
    else:
        minus[parameter] -= delta
        plus[parameter] += delta
        mode = "central"
    return minus, plus, mode
```

Default delta is .02 in z. An interior pair has z-minus=z_hat-.02 and z-plus=z_hat+.02, so its span is about .04. Near a boundary one endpoint is the original prediction and the other moves .02; the span is about .02. Never clip an endpoint but pretend the span is still .04.

diagnostic_batch.measure (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostic_batch.py:31`) checks the other 12 readbacks remain exactly unchanged and computes `S_j = D(render_minus,render_plus)/(actual_z_plus-actual_z_minus)`. It reuses the base render at a boundary. The engine-readback denominator handles small wrapper rounding differences. The source prediction is verified unchanged.

S is an unsigned measure of local activity. It is not a signed derivative, a gradient of target loss, a probability, a JND, or proof that the parameter is uniquely recoverable. Low S may occur with either a correct or an incorrect setting. High S may occur even when several settings compensate for one another.

The prediction is **not optimized** during this step. Even if a perturbed version is closer to the target, it is not adopted. Otherwise a calibration improvement could be confounded by changing the patch being judged.

Why not perturb every corpus clip?

Central diagnostics need up to `1 + 2*13 = 27` renders per clip, excluding audits. Doing this for 131,072 clips would require up to 3,538,944 renders. Instead, frozen subsets provide bounded experiments: 128 smoke, 2,048 calibration and 512 validation. The 2,560 calibration/validation cases alone need up to 69,120 renders before boundary reuse and audits. Derived WAVs are normally streamed and discarded; scores/readbacks/hashes are saved. Only selected demonstration audio is retained.

Audits on the first four predetermined ranks of each batch check exact repeated rendering, exact known-parameter roundtrip, inactive ENV2 attack at zero depth and unchanged predictions. Passing these checks supports implementation correctness for the tested fixtures, not a global identifiability proof.

14. Was the .02 perturbation choice checked?

Yes. The bounded study used the first 256 preassigned calibration ranks, kept the same predictions and compared .01/.02/.05. It reused 256 saved .02 cases and rendered 512 extra cases for .01/.05. Eight additional renderer audits passed.

stability.select_pilot (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/stability.py:37`) checks the calibration-only, aligned, fixed selection. agreement (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/stability.py:80`) compares ranks and log1p magnitudes; summarize (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/stability.py:97`) separates all/common-interior/boundary-affected cases.

Most controls have very similar rankings. Wave frame and bend are more step-dependent; amplitude attack shows magnitude changes even with high rank agreement. Some boundary groups are tiny. No post-hoc pass cutoff or replacement step was selected. Default .02 and existing calibrators stay unchanged. Detailed numbers are in the metrics reference and step_stability_v1/RESULTS.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/step_stability_v1/RESULTS.md`).

Back to contents

Part 6. Reliability calibration

15. How does U/S/R become a probability q?

We need examples where the answer is known. For calibration clip i and parameter j, compare predicted z with original z. Call it correct when the error is <=.05. That gives a binary teaching label y, separate from the continuous inverse target.

c2_estimation/v4/calibration.py:39 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:39`)

```
def labels(frame,name,tau):
    error = np.abs(frame[name+"_predicted_z"].to_numpy(dtype=np.float64)-frame[name+"_actual_z"].to_numpy(dtyp
        e=np.float64))
    if not np.isfinite(error).all():
        raise ValueError("Invalid correctness labels")
    return (error<=tau).astype(float),error
```

The primary event is recovery within 5% of the **restricted parameter interval**. It is not a sound-quality threshold. Secondary .025/.10 tolerance analyses check whether conclusions depend on this one event definition.

For each parameter, a small regularized logistic model learns the relation between deployment-available measurements and that correctness label:

```
U -> ln(max(U,1e-12)); S -> ln(1+S); R -> ln(1+R)
standardize each feature using calibration-only mean/std
q = sigmoid(intercept + weight_U*scaled_U + weight_S*scaled_S + weight_R*scaled_R)
```

features (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:29`) implements transforms; unused terms are omitted for each ablation. The logarithms compress skewed magnitudes; calibration-only scaling puts features on comparable scales without leaking validation information into fitting statistics.

c2_estimation/v4/calibration.py:46 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:46`)

```
def fit(x,y,c):
    """Mean binomial NLL + ||w||²/(2*C*n); intercept unpenalized; no class weights."""
    x,y = np.asarray(x,dtype=np.float64),np.asarray(y,dtype=np.float64)
    if x.ndim != 2 or len(x) != len(y) or not len(y) or c<=0 or not np.isfinite(x).all() or not
        np.isin(y,[0.,1.]).all():
        raise ValueError("Invalid logistic calibration input")
    mean,std = x.mean(axis=0),x.std(axis=0)
    std = np.where(std<1e-12,1.,std)
    model = {"mean":mean.tolist(),"std":std.tolist(),"C":c,"count":len(y),
             "success_count":int(y.sum()),"class_weight":"none","constant_probability":None}
    if np.unique(y).size == 1:
        model.update(constant_probability=float(y[0]),weights=[0.]*x.shape[1],intercept=0.,solver_status="sing
            le_class_constant")
```

```
        return model
    standardized = (x-mean)/std
    n = len(y)
    def objective(parameters):
        w,b = parameters[:-1],parameters[-1]
        logits = standardized@w+b
        penalty = np.dot(w,w)/(2*c*n)
        loss = np.mean(np.logaddexp(0,logits)-y*logits)+penalty
        residual = expit(logits)-y
        gradient = np.r_[standardized.T@residual/n+w/(c*n),residual.mean()]
        return loss,gradient
    initial = np.r_[np.zeros(x.shape[1]),np.log(y.mean()/(1-y.mean()))]
    result = minimize(objective,initial,jac=True,method="L-BFGS-B",
                      options={"maxiter":1000,"ftol":1e-12,"gtol":1e-8})
    if not result.success or not np.isfinite(result.x).all():
        raise ValueError("Logistic fit failed: "+str(result.message))
    model.update(weights=result.x[:-1].tolist(),intercept=float(result.x[-1]),
                 solver_status=str(result.message),objective=float(result.fun))
    return model
```

`fit` uses 2,048 calibration clips, SciPy L-BFGS-B and mean binomial NLL plus `||w||^2/(2*C*n)`. The intercept is unpenalized. There is no class balancing because we want probabilities under the actual sampled correctness prevalence. A single-class calibration target returns its constant observed probability rather than pretending a discriminative model can be estimated.

For each parameter and variant, C is selected from .01/.1/1/10 using lowest Brier on the separate 512 reliability-validation cases; smaller C breaks ties. This penalty is stronger for smaller C and weaker for larger C. This selection appears inside calibration.run (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:165`). Those validation scores are therefore development estimates, not an unbiased final test. Per tolerance, there are 13 x 4 selected feature-based models plus 13 constants.

If q=.8, the intended interpretation is that similarly scored in-domain predictions should be within tolerance about 80% of the time when calibration holds. It is not a guarantee about one example. U=.02 cannot be read as 98% confidence, and a small R cannot be converted directly to q.

The fitted models, scalers, feature choices and C values are saved in calibrators.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/calibration_tau_0.05/calibrators.json`). No Accept/Approximate/Abstain thresholds are validated yet. The viewer displays continuous probabilities and known reference outcomes only when requested.

Back to contents

Part 7. Evaluation and validation

16. How do we test the research hypothesis fairly?

Use the **same frozen predictions, original settings, audio IDs and tolerance** for all methods. Change only the reliability inputs.

Method	Inputs	Purpose
g0	Calibration success rate for each parameter	Constant-probability reference
Raw-U ranking	U, with high U indicating likely failure	Tests raw uncertainty ranking; has no Brier/ECE probability score
g(U)	U	Main uncertainty-only calibrated baseline
g(U,S)	U and S	Tests adding local sensitivity

Method	Inputs	Purpose
g(U,R)	U and R	Tests adding reconstruction residual
g(U,S,R)	U, S and R	Full proposed feature set

Comparing only raw U against the full calibrator would mix the effect of calibration with the effect of new diagnostics. g(U) controls for calibration. Comparing the full model against g(U,R) asks more specifically whether S adds value given R.

Primary development macro Brier:

Model	Macro Brier	Macro ECE	Macro AURC
g0	0.138242	0.040681	0.568125
g(U)	0.116267	0.037460	0.462585
g(U,S)	0.115908	0.036614	0.459520
g(U,R)	0.115572	0.035921	0.462154
g(U,S,R)	0.115385	0.035282	0.459777

The full method is slightly better in Brier and ECE, but g(U,S) has slightly lower macro AURC. A method need not win every metric. The full-minus-g(U) Brier difference is -.000882, paired 95% interval [-.001684,-.000089]. Against g(U,R), it is -.000187, interval [-.000757,.000400], which includes zero. An exploratory analysis excluding wave frame removes the overall gain over U alone. That analysis explains a pattern; it does not replace the original 13-control endpoint.

Why paired bootstrap and why resample whole audio?

Imagine drawing the evaluated audio examples again with replacement many times. For each resample, compare both methods on exactly the same examples. Keep each clip's 13 controls together because they share a sound and are not independent trials.

c2_estimation/v4/calibration.py:149 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:149`)

```
def paired_brier_bootstrap(y,probabilities,reference="g(U)",replicates=2000,seed=SEED):
    if replicates<1:
        raise ValueError("Positive bootstrap count required")
    indices = np.random.default_rng(seed).integers(0,len(y),size=(replicates,len(y)))
    reference_loss = np.square(probabilities[reference]-y).mean(axis=1)
    result = []
    for variant,q in probabilities.items():
        loss = np.square(q-y).mean(axis=1)
        differences = (loss-reference_loss)[indices].mean(axis=1)
        result.append({"variant":variant,"reference":reference,"metric":"macro Brier difference
            (variant-reference)",
"point_difference":float((loss-reference_loss).mean()),"ci95_low":float(np.quantile(differences,.025)),
            "ci95_high":float(np.quantile(differences,.975)),"bootstrap_unit":"whole audio (13 controls
                together)",
            "replicates":replicates,"seed":seed})
    return result
```

There are 2,000 replicates, seed 20261013; the interval uses the 2.5th and 97.5th percentiles. These intervals condition on the fitted models and validation selection. They do not include all training-seed uncertainty or establish final generalization. Per-control follow-up intervals are exploratory and unadjusted for multiple comparisons.

audit_results.comparisons (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/audit_results.py:63`) adds the conditional U+R comparison and exploratory wave-frame analysis. All measured tables and figures are preserved in results_audit_v1/summary.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/results_audit_v1/summary.json`).

Back to contents

Part 8. Real example and saved evidence

17. A complete real example

Use the viewer's first example, **restricted_bend_sustain_v4_63764**. It is the first preassigned reliability-validation rank, not selected because it sounds good. Its target/reconstruction R is **.098242** (rounded; exact value is in the evidence). Known-parameter rendering reproduces the target samples exactly before playback encoding.

Parameter	Original z	Predicted z	Abs error	Correct at .05?
wave_frame	0.833818	0.820435	0.013383	yes
bend_amount	0.406764	0.572309	0.165545	no
filter_cutoff	0.369066	0.371036	0.001970	yes
filter_resonance	0.087807	0.285323	0.197517	no
filter_drive	0.324894	0.555184	0.230290	no
amp_attack	0.765073	0.758939	0.006135	yes
amp_decay	0.497350	0.447830	0.049520	yes
amp_sustain	0.591115	0.598996	0.007881	yes
amp_release	0.734582	0.737857	0.003276	yes
filter_env_amount	0.647768	0.629890	0.017878	yes
filter_env_attack	0.298005	0.386659	0.088654	no
filter_env_decay	0.419505	0.490898	0.071393	no
filter_env_sustain	0.653288	0.605458	0.047830	yes

Parameter	U in z squared	S	q: U only	q: U+S+R
wave_frame	0.002878	7.507775	0.749571	0.899550
bend_amount	0.050639	0.301070	0.057461	0.078736
filter_cutoff	0.001301	1.769388	0.883703	0.893669
filter_resonance	0.034706	0.662503	0.186259	0.173517
filter_drive	0.078118	0.243207	0.114688	0.131774
amp_attack	0.000109	3.171842	0.996566	0.997187
amp_decay	0.003868	0.438037	0.784666	0.806706
amp_sustain	0.001014	2.195536	0.915876	0.955496
amp_release	0.000103	0.536864	0.999639	0.999618
filter_env_amount	0.031132	0.242141	0.231999	0.200600
filter_env_attack	0.048719	0.017692	0.192919	0.193151
filter_env_decay	0.074211	0.041242	0.107636	0.109026
filter_env_sustain	0.026830	0.246319	0.284738	0.296764

For cutoff, predicted z is about .3710 and original z .3691, so error is about .0020 and y=1 at .05. The minus/plus settings are about .3510/.3910, the measured span about .04, S about 1.7694 and full-model q about .8937. Those are different measurements with different meanings. q is not calculated by simply subtracting error from one; original error would be unavailable for a new user recording.

This same clip has a small whole-audio R but several parameters outside tolerance. Filter-envelope amount is within tolerance despite a low q. An individual low-q success is not, by itself, evidence of miscalibration. Calibration concerns frequencies across many similarly scored predictions.

Open example.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/viewer_v1/examples/restricted_bend_sustain_v4_63764/example.json`), target.wav (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/viewer_v1/examples/restricted_bend_sustain_v4_63764/target.wav`), predicted.wav (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/viewer_v1/examples/restricted_bend_sustain_v4_63764/predicted.wav`) and spectrograms.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/viewer_v1/examples/restricted_bend_sustain_v4_63764/spectrograms.png`). Playback copies are 16-bit WAV for browser compatibility; numerical diagnostics use canonical float32 samples before quantization. No independent gain matching is applied.

18. What EDA and scientific checks were actually saved?

EDA means looking at the data before claiming the model or experiment is sound. It includes distribution, range, quality and error checks, not just an attractive plot.

Evidence	Question answered	File to show
Distributions and split balance	Are allowed values covered, and do non-test groups look comparable?	target_distributions_and_split_balance.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/target_distributions_and_split_balance.png`)
Parameter correlation	Does the sampled training design strongly correlate controls?	training_parameter_correlation.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/training_parameter_correlation.png`)
Audio QA	Do fixed training examples have valid rate, duration, level, DC and hashes?	training_audio_quality.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/training_audio_quality.png`)
Learning curves	What happened during training and checkpoint selection?	learning_curves.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/learning_curves.png`)
Actual vs predicted	Which parameters are recovered well or poorly?	validation_actual_vs_predicted.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/evidence_ensemble/validation_actual_vs_predicted.png`)
Error across values	Are errors concentrated in parts of a parameter's interval?	validation_error_by_actual_value.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/validation_error_by_actual_value.png`)
Envelope context	Is recovery different at weak versus strong generating depth?	filter_envelope_error_context.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/filter_envelope_error_context.png`)
Raw uncertainty vs error	Does U visually track error before calibration?	raw_uncertainty_vs_squared_error.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/evidence_ensemble/raw_uncertainty_vs_squared_error.png`)

Evidence	Question answered	File to show
Reliability diagrams	Do predicted probabilities match observed success frequencies?	reliability_diagrams.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/calibration_tau_0.05/reliability_diagrams.png`)
Matched coverage	Does a method retain more useful predictions at the same fraction?	matched_coverage_audit.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/results_audit_v1/matched_coverage_audit.png`)
Step robustness	Do sensitivity ranks and magnitudes depend on delta?	step_size_magnitude_agreement.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/step_stability_v1/step_size_magnitude_agreement.png`)

The baseline evidence pack has 10 figures; the ensemble pack has 11. Each has underlying CSV tables and provenance. The correlation maximum .004659 describes sampling relationships, not acoustic identifiability. Context plots using true depth are descriptive EDA, not deployment features or causal effects.

The latest saved CPU suite discovered 43 tests: 41 passed, two CUDA-only checks skipped. The CUDA-only checks passed in an earlier dedicated GPU record. These tests check implementation, including split protections, restart reproducibility, Gaussian variance arithmetic, calibration metrics, perturbations and audio exports. They do not test the scientific hypothesis against human listeners.

Open verification_validation_viewer_v1/verification.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/verification_validation_viewer_v1/verification.json`), verification_gpu_v1/verification.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/verification_gpu_v1/verification.json`) and RESEARCH_EVIDENCE.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/RESEARCH_EVIDENCE.md`).

Back to contents

Part 9. Technology choices and justifications

19. Why these technologies and not something larger?

Choice	Justification	Limitation / alternative
Synthetic Vital pairs	Exact generating labels and a controllable forward engine	Restricted domain; not representative of every real recording
One fixed preset family	Keeps the reliability question experimentally manageable	Limits external validity; multi-oscillator symmetry is not the main tested mechanism
Sobol in 13 coordinates	Reproducible structured coverage with a power-of-two design	Not uniform perceptual coverage
64-band log-mel + small PyTorch CNN	Compact, established audio representation; affordable independent runs	Best possible inverse architecture is unproven
Gaussian deep ensemble	Gives input-dependent variance and between-model spread	More compute; variances need validation/calibration
Multi-resolution STFT diagnostic	Transparent timing/frequency comparison; easy to fix and reproduce	Its scale is not a perceptual probability
Regularized logistic models	Small, inspectable and controlled across feature ablations	Restricted functional form; may miss interactions
NumPy/SciPy	Deterministic transforms, numerical distances and optimization	Numeric policies and versions still matter
pandas/PyArrow	ID-aligned manifests, compact Parquet, readable result CSVs	File joins must preserve IDs and label precision
SoundFile	FLOAT masters and PCM16 playback export	Playback quantization differs from diagnostic samples
Matplotlib	Saved scientific figures with underlying data	A plot alone does not establish the hypothesis
HTML/CSS/JavaScript	Simple local saved-result viewer; no build service needed	No live inference backend connected yet

The actual recorded runtime is Python 3.13.14, Vita 0.0.5, NumPy 2.5.1, SciPy 1.18.0, pandas 3.0.5 and SoundFile 0.14.0. Training used the isolated `.venv-c2` PyTorch 2.13.0+cu130 environment with CUDA 13.0 on the RTX 4060 Laptop GPU. The original `venv` remains CPU PyTorch 2.13.0+cpu and retains synthesis dependencies. These are recorded local versions, not recommendations to install the latest release.

CLAP is not currently used. Its audio-language training objective motivates a possible learned-embedding comparison, but does not establish sensitivity to tiny changes in these controls. That would require a controlled comparison using the same predictions, perturbations and partitions. See the original CLAP paper. MFCC is also not this C2 inverse input or diagnostic. Log-mel and MR-STFT have different jobs; choosing both does not mean arbitrarily averaging unrelated scores.

Back to contents

Part 10. Revision and code reference

20. Questions you should be able to answer

What is the novelty? Testing whether forward-rendered R and local S improve per-control correctness reliability of one frozen inverse estimate over calibrated uncertainty alone. The individual building blocks already have prior work.

Why not just improve the CNN? That is useful but answers a different question. Holding its outputs fixed lets us isolate whether the diagnostic inputs add useful reliability information. The current network is a practical experimental baseline.

Why not use the true settings to compute S? They are unavailable for a new user clip. S is computed around the delivered prediction, just as it would be at deployment. Truth is used for supervised targets, correctness labels and evaluation only.

Does high S mean low uncertainty? No fixed rule is assumed. An active knob can still be ambiguous or wrongly estimated. The relation is learned separately for each parameter, then compared against simpler reliability models.

Why is R the same input for every parameter? It measures the complete patch's audio mismatch. Each calibrator combines that shared context with its own U and S.

Why not use .05 as a universal confidence cutoff? .05 is the allowed parameter error in z, not a probability threshold. q thresholds for accepting or abstaining are a separate unresolved decision and must be fixed before final test.

Why does an envelope parameter become hard to recover? At zero envelope depth, its timing cannot affect the filtered sound. At sustain 1, the decay level change can disappear. Other controls and parameter interactions can also mask effects. These are motivations; present observed data rather than claiming every error has one proven cause.

Did S prove useful? Development full-model Brier is slightly lower than U-only, but adding S to U+R has an interval containing zero. Much of the gain is concentrated in wave frame. The final test has not been run, so the strongest justified answer is that the added benefit of S is currently uncertain.

Is Brier only calibration? No. It scores the quality of probability forecasts, including their discrimination and calibration. Reliability diagrams/ECE inspect agreement with observed frequencies; ranking and selective metrics examine other roles.

Why not claim 90% sound similarity from q=.9? q concerns recovery of a generating parameter within tolerance. Sound discrepancy R and human perception are different quantities. A new audio clip has no original setting available to calculate its MAE.

Can I claim the component is finished? The development research pipeline and saved demonstration are complete. Live input/export, final method locking, untouched test evaluation and the later human study remain. Be precise about that boundary.

What does the small human study add? It can test whether users understand and find the display useful while inspecting/editing patches. It does not replace known-parameter calibration evaluation or automatically validate D as perceptual similarity.

Does C2 require C3? No. Alternative-patch retrieval/reranking belongs to C3. C2 can assess its original predicted patch independently. A shared integrated interface can combine components later without changing this controlled experiment.

How would the live application work? Validate an input clip against supported conditions, apply the fixed preprocessing, run frozen members, render the predicted patch, compute R/S, apply saved calibrators and export patch/audio/scores. Vita already serializes presets in the generation tooling; the C2 live service and export path still require implementation and reload verification.

21. One-minute explanation to practise

> We generated a controlled dataset by setting 13 Vital parameters, recording what the synth actually accepted, and rendering each sound under the same note and recording conditions. I use those pairs to train a small CNN on log-mel features. Three probabilistic CNNs provide a predicted patch and raw parameter uncertainty. I then freeze the patch. Rendering it gives reconstruction error R, and changing one parameter at a time gives local sensitivity S. Separate calibration data teaches small models to estimate whether each setting is within the declared tolerance. I compare U-only, U+S, U+R and U+S+R on the same predictions. Development results show a small overall probability improvement, but sensitivity's extra benefit is not yet established. The test set stays reserved until the final method is locked. The current page shows saved examples; live upload/export is next.

Use this to understand the argument, then put it into your own words. If asked for a number, state its partition, sample count, coordinate and metric before interpreting it.

22. File map and validation pipeline

The workflow coordinator is run_development.run (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/run_development.py:28`). After ensemble completion, it runs ensemble EDA, smoke prediction/diagnostics, calibration prediction/diagnostics, reliability-validation prediction/diagnostics, then calibrations at .05, .025 and .10. Stages verify completed outputs before reuse. Per-case diagnostic files permit resume; source/signature drift is rejected. The resulting status.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/status.json`) records completion. A stale historical running flag or open terminal alone would not prove a process is still progressing.

Source	Main blocks and responsibility
generation/vital_setup.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/vital_setup.py`)	Profile loading, engine/control validation and base-preset building
generation/scripts/05_generate_pilot_dataset.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py`)	Frozen Sobol sampling, setters/readbacks, render call, QA and versioned pilot generation
generation/generate_corpus.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py`)	Full-corpus recipe guard, parallel fresh renders, raw masters, checkpoint/resume and consolidation
generation/audio_processing.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audio_processing.py`)	Fixed DC policy and separate optional review-level helpers
generation/corpus_audio.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/corpus_audio.py`)	Raw loading, canonical once-only conditioning and sample-hash checks
generation/audit_full_corpus.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audit_full_corpus.py`)	Full file/label/QA/manifest verification and optional reference export
generation/audit_control_units.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audit_control_units.py`)	Normalized/native/display and preset serialization checks
c2_estimation/pipeline.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py`)	Reused mel/CNN/determinism primitives; its command-line main is the historical eight-control run
v4/data.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py`)	Exact label/reference validation, coordinate mapping, frozen splits and subsets
v4/features.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/features.py`)	Guarded inverse-feature extraction, shards, cache loading and train statistics
v4/baseline.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/baseline.py`)	Single CNN fitting, checkpoints, recovery metrics and training plots
v4/compare_baselines.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/compare_baselines.py`)	Earlier/full baseline comparison on the same original 512 IDs
v4/setup_cuda.py (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/setup_cuda.py`)	Isolated training environment and runtime record; not inference logic

Source	Main blocks and responsibility
v4/ensemble.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py)	Gaussian heads, NLL fitting, independent seeds and mixture U
v4/predict_subsets.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/predict_subsets.py)	Frozen inference on preassigned non-test subsets
v4/diagnostics.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py)	Renderer, STFT representation/distance, perturbation rule and worked example
v4/diagnostic_batch.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostic_batch.py)	Many fixed predictions -> measured R/S, audits, case resume and readback exports
v4/calibration.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py)	Correctness labels, logistic fitting, C selection, ablations and reliability metrics
v4/evidence.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/evidence.py)	EDA/error/uncertainty figures, tables and provenance
v4/audit_results.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/audit_results.py)	Per-control audit, conditional/paired comparisons and coverage summaries
v4/stability.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/stability.py)	Fixed calibration-only .01/.02/.05 robustness study
v4/verification.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/verification.py)	Saves synthetic test log/counts/source fingerprints
v4/verify_forward_batch.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/verify_forward_batch.py)	Checks new batch diagnostics against a saved real worked example
v4/build_viewer.py (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/build_viewer.py)	Fixed example selection, verified audio/data export and display refresh
viewer/index.html (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/viewer/index.html), app.js (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/viewer/app.js), style.css (path: D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/viewer/style.css)	Browser layout, saved-data interactions and styling

The test files are under `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/tests/`: `test_v4.py` checks data/model/restart behaviour; `test_ensemble.py` checks heads, mixture moments and reproducibility; `test_reliability.py` checks diagnostic and calibration arithmetic; `test_stability.py`, `test_audit_results.py` and `test_viewer.py` cover the added checks. `test_pipeline.py` preserves pilot checks.

Do not confuse model weights, patches and audio

File format	What it contains
`.py`	Instructions implementing the method
`.pt`	Learned PyTorch model tensors/checkpoint metadata; not a Vital preset
`.vital`	Complete synthesizer preset state, including fixed configuration/assets
`.wav`	Audio samples rendered or recorded from a patch
`.parquet` / `.csv`	ID-aligned parameter, prediction or metric tables
`.npy` / `.npz`	Numeric feature arrays or ensemble member moments
`.json`	Configurations, labels/diagnostics, fitted calibrators, statuses or hashes
`.png`	A visualization of measured data

The eventual patch download must be a full preset with the fixed setup plus the 13 predicted controls, not simply a renamed model checkpoint or parameter vector. Reload and audio/readback checks must verify the export before presenting it as usable.

Proposal alignment

The supplied project report (path: `E:/SLIIT/Projects/RESEARCH PROJECT/Audio to Synth/1. Project Progression/1. Project Proposal/Individual Reports/IT23203280_Acoustic_Identifiability_Aware_Reliability_Calibration.pdf`) was checked at methodology pages 17-24 and architecture/responsibility pages 28-29. It is a design reference, not the authority for current completion claims. Current source, protocol and saved measured outputs establish implemented counts and status. The individual reliability experiment is distinct from the team's shared generation infrastructure and from C1/C3/C4 research contributions.

23. Small dictionary for reading the code

Term or symbol	Meaning here
x / x_hat	Original target waveform / waveform rendered from predicted settings
theta / z	Parameter vector; this implementation usually computes in research z
j / e_j	Parameter number / direction that changes only that parameter
delta	Small parameter movement used for the acoustic check
tau	Maximum absolute parameter error counted as correct
y / q	Known binary correctness label / estimated probability of that label
Seed	Starting state that reproduces a pseudorandom sequence under the same implementation
Sobol dimension	One varying control in the generated design
Sample ID	Stable link between one rendered clip, its settings and later results
Manifest	Table saying which samples exist and where their audio/labels belong
float32 / float64	Two numeric precisions; waveform/features/training use float32, many reporting/readback calculations retain float64
Hash / SHA-256	Fingerprint for detecting changed data or source bytes
FFT / STFT	Frequency analysis of one window / repeated analysis through time
Hann window	Smooth weighting on each analysis frame to reduce edge discontinuities
Mel band	A triangular frequency grouping used to compact the spectrum
Log / dB	Scale compression; dB power uses 10*log10, while diagnostic LM uses natural log magnitudes
Convolution	A learned small pattern detector applied across the input
ReLU	Keeps positive activations and sets negative ones to zero
Pooling	Combines nearby feature positions to reduce array size
Dense layer	Combines all supplied features through learned weights
Sigmoid	Smooth function mapping a number into 0-1
Softplus	Smooth positive-valued function, used before adding the variance floor
Batch	A small group processed together; 128 training clips here
Epoch	One training pass through the selected training set
Learning rate	Scale controlling the size of optimizer updates
Backpropagation	Computes how model weights contributed to the training error
Adam	The optimizer that uses those gradients to update weights
Weight decay	Penalty discouraging unnecessarily large model weights
Checkpoint	Saved model state; last checkpoints also retain training-resume state
Inference	Using trained weights to produce a prediction without fitting them
Heteroscedastic	The predicted variance can depend on the input clip

Term or symbol	Meaning here
Variance	Average squared spread; its units are squared coordinate units
Mixture moments	Mean and variance calculated by combining all ensemble members
Logistic calibration	Learning a small probability rule from measurements and known success/failure outcomes
Regularization / C	Constraint on fitted weights; in this calibrator smaller C means stronger penalty
Ablation	Keep the experiment fixed but remove/change one feature set to test its contribution
Leakage	Using protected examples or unavailable truth in a way that makes an evaluation unfairly easy
Coverage / abstention	Fraction kept / decision not to rely on some predictions
Bootstrap	Resample evaluated examples with replacement to describe uncertainty in a summary
Out of domain	A clip outside the recording/synth/parameter conditions represented by the experiment

When a term is unfamiliar, point back to a real example. For instance, the cutoff row in section 17 gives one actual z, U, S, q and correctness label; they are not five names for the same kind of confidence.